Australian National University

Lab One

Getting started with Vivado

Overview

In this lab, you will learn to:

- Create projects and design simple combinational systems in Vivado.
- Review the RTL structure of a design and simulate its logic behaviour, including assessing the potential impact of transition delays on system performance.

You will also start becoming familiar with:

- Hardware description languages (HDL), specifically Verilog.
- Simple combinational logic circuits such as a multiplexer and a 1-bit full adder.
- The concept of testbenches and logic simulation.
- Vivado's simulation environment, ISim.

Table of Contents

Lab Overview 3
Learning Outcomes 3
Activity 1: Creating your first digital design [4 marks – approximately 30 minutes] 3
Background 3
Procedure 4
Step One: Create a project in Vivado 4
Step Two: Create a Verilog design source 6
Step Three: Visualising your design 8
Step Four: Simulating your design 9
Activity 2: Designing a 1-bit Full Adder [8 marks, approximately 30 minutes] 13
Background 13
Procedure 14
Activity 3: Timing hazards in a multiplexer [8 marks, approximately 1 hour] 15
Background 15
Procedure 15
Appendix 19

 © ANU 2025

Lab Overview

Learning Outcomes

By completing this lab, you will learn to:

1. Create projects and handle files in Vivado.
2. Create design sources and describe simple combinatorial logic circuits in Verilog.
3. Identify the essential building blocks of a simulation testbench.
4. Simulate combinatorial designs, including assessing the impact of time delays.

To complete this lab, you need access to the Vivado Design Suite running on Windows or a supported Linux operating system.

Activity 1: Creating your first digital design [4 marks – approximately 30 minutes]

Background

A multiplexer (a.k.a. MUX) is an electronic switch that connects one of several inputs to the output based on the selection signal *S*. Figure 1 describes the simple case of a 1-bit MUX.

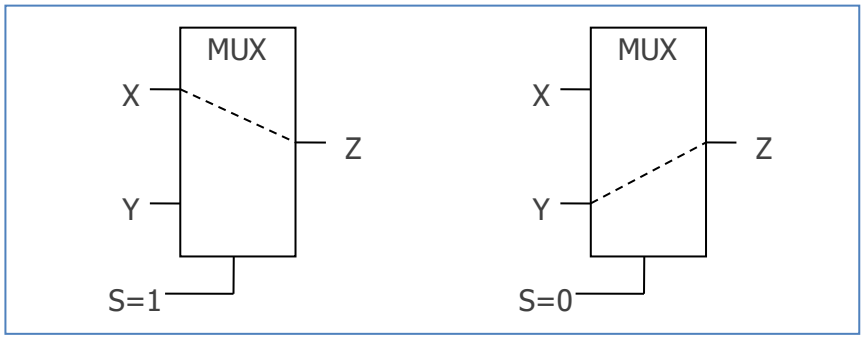


Figure 1: A 1-bit MUX.

The truth table of the 1-bit MUX is shown in Table 1. The 'xx' notation indicates a "don't care" condition. Note that we consider the MUX here as a combinational circuit.

X	Y	S	Z
0	xx	1	0
1	xx	1	1
xx	0	0	0
xx	1	0	1

Table 1 : Truth table of the 1-bit MUX.

The truth table allows us to derive the logic representation of the system. The table can be immediately translated into a Boolean expression, which can in turn be rendered into a schematic or Verilog HDL code.

 © ANU 2025

Next, we derive a minimal Boolean logic representation of the MUX using a Karnaugh Map. Table 2 shows the Karnaugh Map of the 1-bit MUX (notation "X" means X=1).

S \ XY	00	10	11	01
0	0	0	1	1
1	0	1	1	0

Table 2: Karnaugh Map of the 1-bit MUX.

From Table 2, we can derive the Boolean logic expression

```
Z = (X & S) | (Y & (~S))
```
(1)

where ~S means NOT(S). The corresponding schematic diagram is shown in Figure 2.

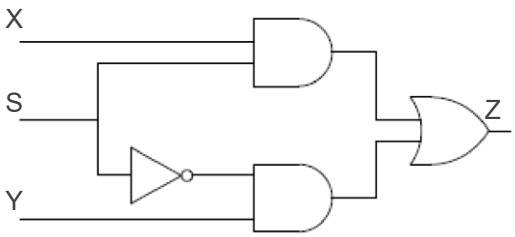


Figure 2: Schematic description of the 1-bit MUX.

Procedure

Step One: Create a project in Vivado

1. Launch Vivado and create a new project.

Figure 3: Vivado's Quick Start menu.

Name your project: 'my_mux_project'. The project name should only contain letters, numbers and underscores. Please avoid using white space and other special characters as they have been known to trigger processing errors.

2. Select the directory in which to store the project's files. It is recommended that you create a dedicated FPGA development directory on your computer to organise the files. Please do not choose OneDrive locations as working folders – OneDrive paths are very long and can create file handling problems in Vivado. If you are using the lab computer, we recommend working locally and transferring files out at the end of the session, for example using a

 © ANU 2025

USB stick or cloud upload. Please do not use white spaces or special characters in the project directory name. It is recommended that you only use letters, numbers and underscores.

3. Make sure the "create subdirectory for your project files" box is checked. Then click Next.
4. Select your Project Type as RTL Project.

   Check the box: "Do not specify sources at this time". You don't currently have any source files to specify.
5. You may be asked to select the default file type and language for your project's source files. This is set to Verilog by default.
6. Specify the Default Part. This refers to the specific chip you intend to ultimately use with your design, which will be different depending on the specific development board you are using. The ANU labs are equipped with Zybo Z7-20 boards. If you wish to practice at home, you are encouraged to borrow a Basys3 board. Information in the manuals will primarily cover settings the Zybo Z7-20, however, we will also provide an Appendix with alternative configuration options for the Basys3 board. This applies to all FPGA labs.

   Zybo Z7-20 utilises a Xilinx Zynq-7000 family SoC (system on a chip), which integrates a dual-core ARM Cortex-A9 processor and a 28 nm Xilinx FPGA logic in a single device. For now, you will not be using the processor in the SoC, so you can treat the SoC device as a standalone FPGA.

   Basys3, on the other hand, utilises an Artix-7 FPGA.

   You can search for your part number directly using the search bar or narrow the list of available options by selecting the appropriate options for your development board (see Appendix for alternative settings if using a Basys3).

Development Board	**Zybo Z7-20**
SoC Family	Xilinx Zynq-7000 family
Package	CLG400
Speed grade	-1
Temperature grade	All Remaining
Part number	xc7z020clg400-1

Caution: It is crucial that you select the correct part number for your project. If you select the wrong part, your design will not successfully deploy to the chip.

 © ANU 2025

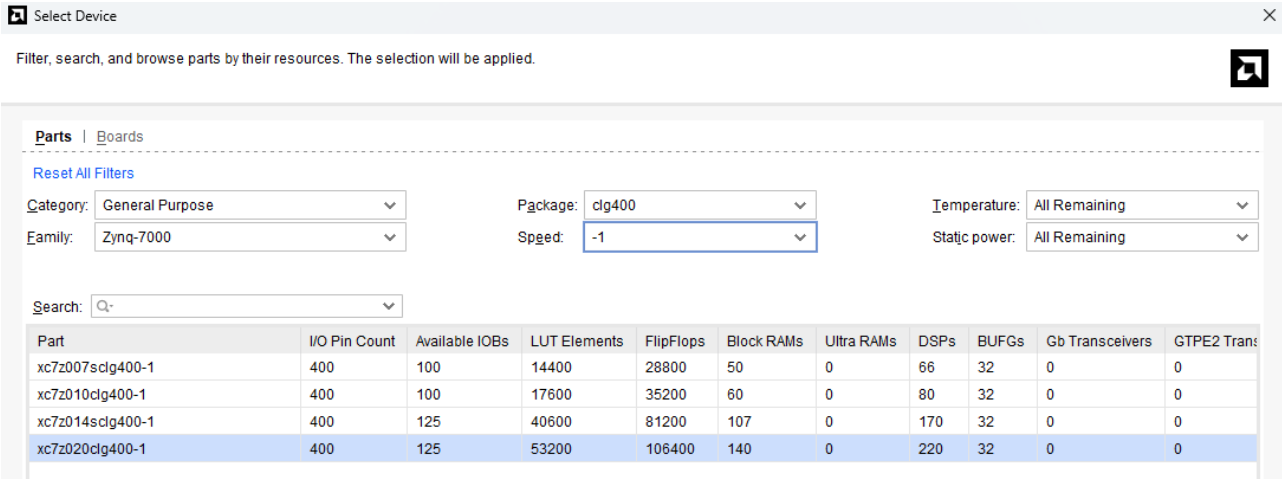


Figure 4: Default Part selection window highlighting the SoC for Zybo Z7-20.

7. When you've selected your Default Part from the list, click Next.
8. Review the information you've specified in the previous steps. Click Finish. The Vivado Integrated Design Environment (IDE) will now appear.

Step Two: Create a Verilog design source

1. At the top left of the Vivado window, click on the 'Add Sources' button.
2. You will be presented with three options: to add or create constraints, design sources, or simulation sources. Select design sources and click Next.
3. There are three options for you to choose from: add files, add directory, and create file. Select the create file option.
4. Select the file type as Verilog; name the file as 'my_mux'; set the file location to <Local to Project>, which is the default option. Once again, make sure you use only letters, numbers and underscores in the file name.

 © ANU 2025

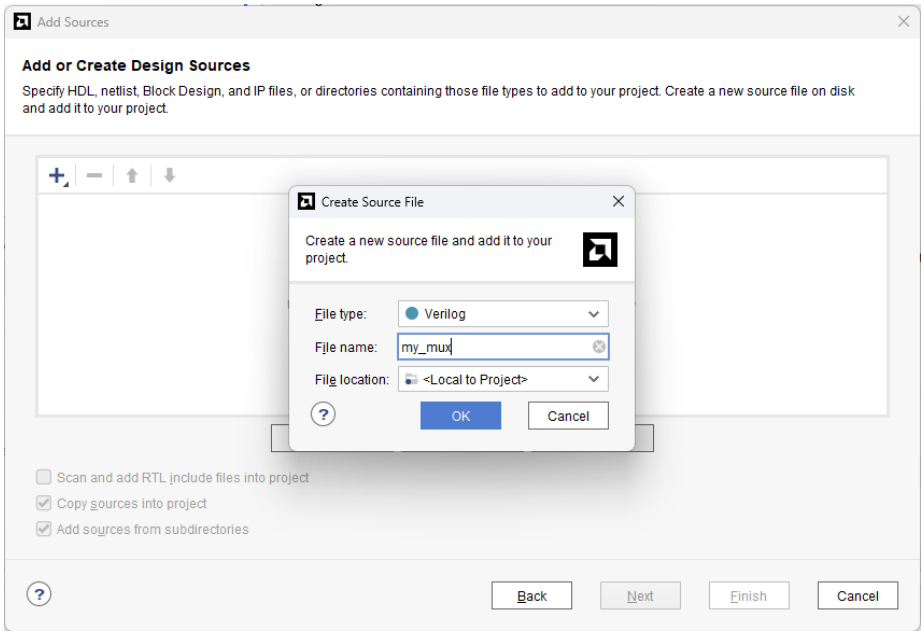


Figure 5: Creating a new design source file in Vivado.

5. Vivado gives you the option to define your module's I/O properties, like in Figure 6. **You can skip this step** because in bigger projects, it could be difficult to determine all I/O at the beginning. Simply Click OK to skip through.

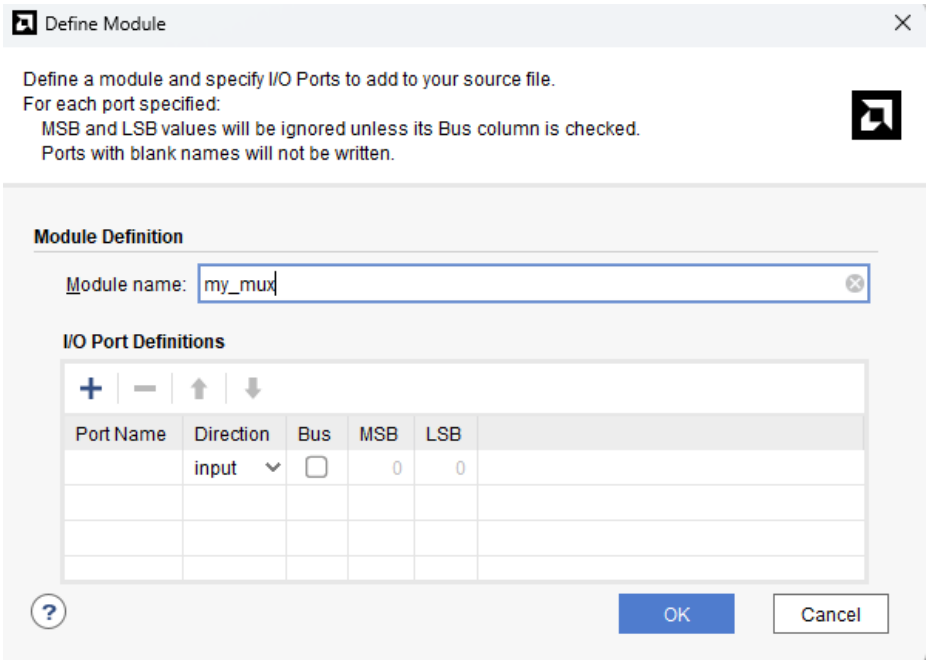


Figure 6: The option of defining module's I/O when creating a new design source.

6. You will now be returned to the Vivado IDE. The file you just created should appear in the 'Design Sources' directory in the Sources pane at the top left of the screen. Double-click on the file 'my_mux.v' to open it.

 © ANU 2025

7. This module needs to describe the logic function that was derived from the Karnaugh map. Declare the port entities as 'wires' and write the `assign` statement, as shown in Code 1. **The keyword "assign" describes continuous circuit connections.**

Code 1

```
`timescale 1ns / 1ps

module my_mux(
    input wire X,
    input wire Y,
    output wire Z,
    input wire S
    );

assign Z = (X & S) | (Y & (~S));

endmodule
```

Compare the logic statement with the Karnaugh map. What we have effectively described here in Verilog is a physical connection between logic gates.

The first line `` `timescale 1ns / 1ps `` is only relevant for simulations.

Make sure the module name is the same as the file name, e.g., the Verilog file 'my_mux.v' should have the module declaration "module my_mux(…)" in it. It is a Verilog language requirement that the file name and the module header are consistent.

Step Three: Visualising your design

You can review the way in which Vivado interprets your code as a system of interconnected basic constructs (gates, registers, etc). This style of system visualisation is called RTL (register-transfer level) framework.

1. Look for "RTL ANALYSIS" on the left-hand pane of the main Vivado window and click "Open Elaborated Design".
2. After some processing (which may fail if you have outstanding errors in your code), you will be presented with a schematic description of your design, as shown in the next figure. Check if the circuit matches the original design intent. (If you make a change to your code, you need to reload 'Elaborated Design' to see the changes in the schematic).

 © ANU 2025

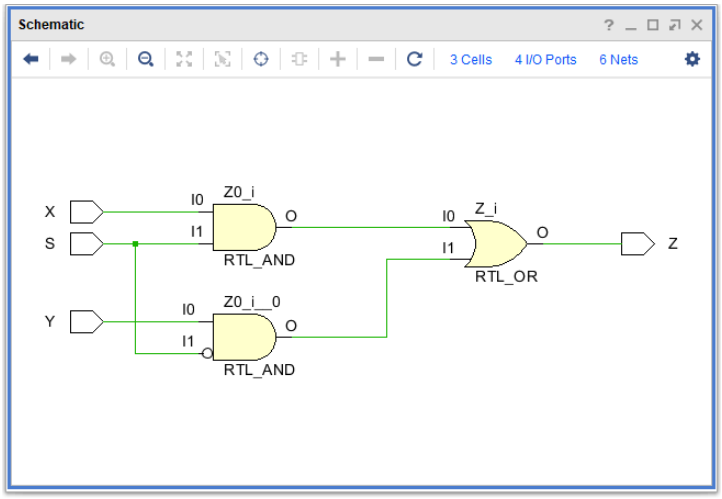


Figure 7: RTL view of the MUX design.

Note: You are not required to view the RTL schematic every time, but you should be aware of this feature as it can be a useful troubleshooting tool.

Step Four: Simulating your design

Simulation requires a test bench, which is a special type of file that describes a virtual operating environment for the design under test. The test bench file specifies what test inputs should be fed to your design, and what output signals should be read and analysed. Simulations are run using the internal Vivado Simulator, which is called ISim.

Note: Test bench files are only used for simulation and have no role in the final hardware implementation.

1. Right click on *Simulation Sources* and select *Add Sources.*

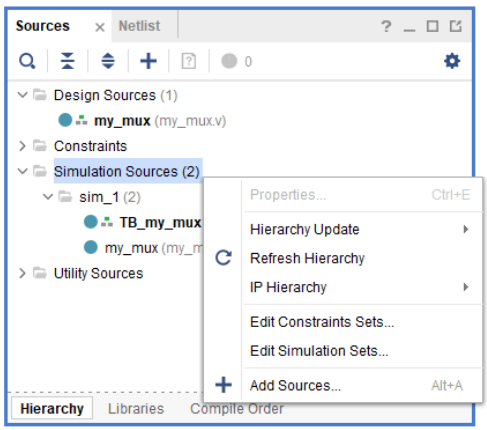


Figure 8: Adding a simulation source.

2. In the window that appears, select *Add or create simulation source*, then click *Next*
3. In the following window, click *Create File* to create a new Verilog source.

 © ANU 2025

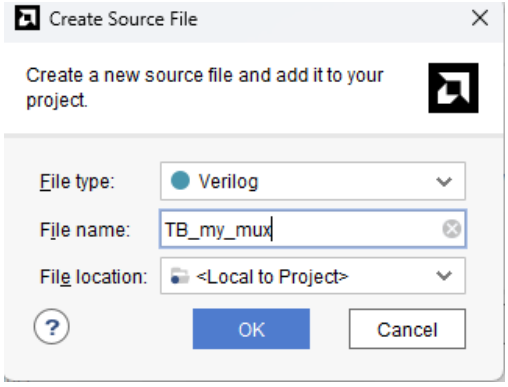


Figure 9: Naming the simulation source.

4. In the next window, do not assign any inputs and outputs to the new module. Just click through.
5. Replace the contents of the test bench file with Code 2 on page 12 This is an example of **instantiation,** where a higher level module (TB_my_mux.v) references a lower level module (my_mux.v). Instantiation syntax is covered in the two lecture slides over the page.

 © ANU 2025

Instantiation

- The hierarchical nature of the Verilog language can be exploited by coding sub-modules separately and then calling upon them (***instantiating***) from a higher-level module. Let's refer to the calling module as the *master (m)* and the called module as the *slave (s)*
  - Instantiation is routine in simulations where the test bench file has the master role and the module under test is *an instance* of the slave
- **Instantiation syntax:**

```
module_name instance_name ( .s1(m1), .s2(m2), … );
```

- Each slave module signal (`s#` stands for slave signal `#`) is listed preceded by a dot, while the corresponding signal in the master module is named in brackets (`m#` stands for master signal `#`).
  - **Paired signals need not have the same name**

Instantiation example

Slave module

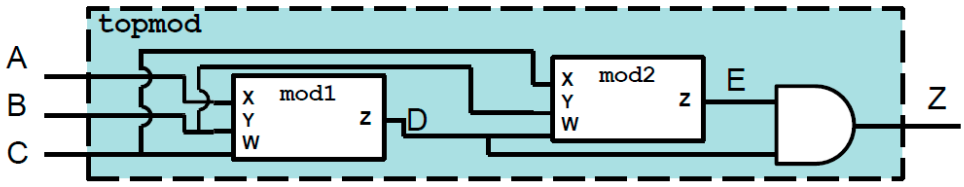
```
module boolmod  (
   input wire X, Y, W,
   output reg Z);

   always @(*)
        Z = (X & Y)|(X & W);
end
endmodule
```

Master module

```
module topmod (
   input wire A, B, C,
   output wire Z);
   wire D, E;
   boolmod mod1(.X(A),.Y(B),
     .W(C), .Z(D));
   boolmod mod2(.X(C),.Y(B),
     .W(D), .Z(E));

   assign Z = D & E;
endmodule
```



© ANU 2025

Code 2

```
`timescale 1ns / 1ps // timestep duration and time precision

module TB_my_mux;

// Inputs to Unit Under Test (UUT)
   reg X;
   reg S;
   reg Y;

// Output to UUT
   wire Z;

// Instantiate the design to be tested - ensure module name match!
   my_mux UUT( //NOTE: the instance name ('UUT' here) can be anything you like
       .X(X),
       .S(S),
       .Y(Y),
       .Z(Z)
   );

// Duration of simulation. Here set at 300 timesteps
   initial begin: stopat
    #300; $finish;
   end

// Signal changes: #xx means enact change after xx timesteps
   initial begin
    X = 1'b0;
    Y = 1'b1;
    S = 1'b1;

    #30X = 1'b1;
    #30S = 1'b0;
    #30Y = 1'b0;

    #30X = 1'b0;
    #30Y = 1'b1;
    #30S = 1'b1;

    #30X = 1'b1;
    #30Y = 1'b0;
    #30S = 1'b0;
   end
endmodule
```

6. Look for the SIMULATION heading in the navigation pane of the main Vivado window, then click on *Run Simulation* → *Run Behavioural Simulation* (Figure 10).

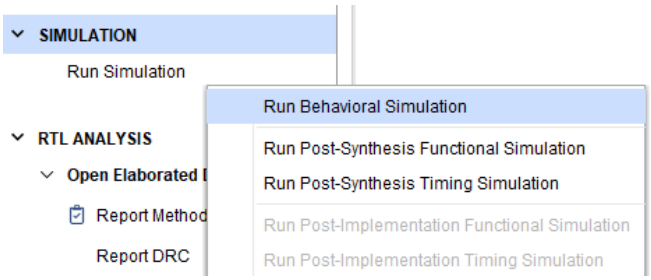


Figure 10: Starting the simulation.

 © ANU 2025

7. Once the computations are complete, the simulator window will open showing the simulation results. Under a tab called *Untitled*, you will see the simulated traces for all signals in the design. Use the Zoom Fit tool to view the whole simulation in one screen. Compare the results with the expected behaviour (i.e., the truth table for the multiplexer). Below is a screenshot of the results (Figure 11).

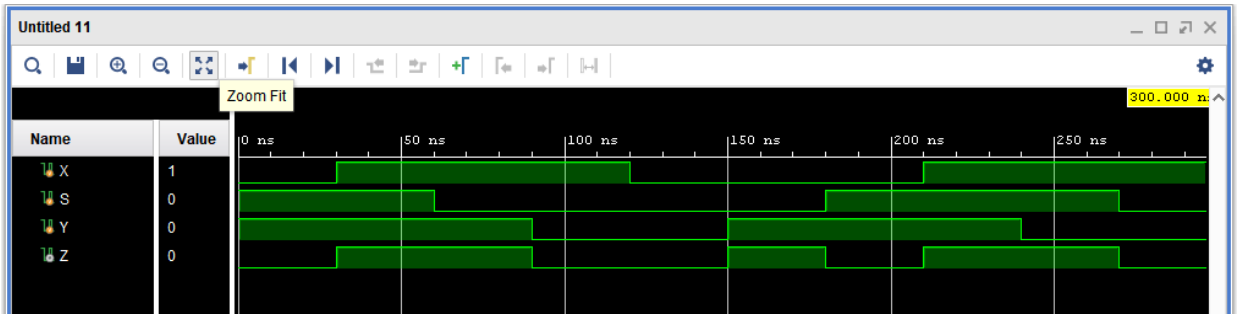


Figure 11: Simulator window showing the simulation results. Note the Zoom Fit button.

8. Copy the results traces in your lab notebook and include some annotations to capture your understanding that the result is indeed correct. You have now completed this task. Don't forget to show your work to the lab demonstrators to receive your grade.

Activity 2: Designing a 1-bit Full Adder [8 marks, approximately 30 minutes]

Background

A full adder is an adder with carry-in and carry-out ports. The truth table of a 1-bit full adder is given in Table 3.

X	Y	C_in	Z	C_out
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Table 3: Truth table of the 1-bit full adder.

 © ANU 2025

Procedure

Your task involves completing the following steps:

1. Derive Boolean expressions for the outputs Z and C_out. Consider XOR gates for Z.
2. Create a new project "my_full_adder". Name the design source file as "my_adder".
3. Complete the source file for the 1-bit full adder based on the derived Boolean expressions.
4. Simulate your design. Please import "TB_Test_Add_1.v" (supplied as a lab resource) as a testbench module. Add a simulation source as in Figure 8. Use the option Add file to import the testbench file.

   Make sure the option "Copy sources into project" is checked. This creates a copy of the imported file in the project folder. If you do not check this box, files will be linked using their original path. This can easily lead to broken dependencies when projects are moved to new locations.

   Check the instantiation syntax. You might need to modify the signal names. Run the behavioural simulation. Figure 12 shows a typical result for a 1-bit adder.

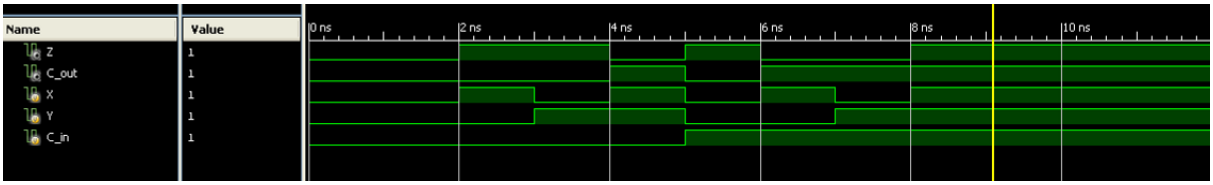


Figure 12: Expected simulation results for the 1-bit adder.

5. Copy your inferred schematic and results traces in your lab notebook and include some annotations to capture your understanding that the result is indeed correct. Show the source file and the simulation results to the lab demonstrators to receive your grade.

 © ANU 2025

Activity 3: Timing hazards in a multiplexer [8 marks, approximately 1 hour]

Background

Simulation as we have done it so far treats the circuits as ideal, i.e., with instantaneous signal changes. However, transition times in digital circuits are nonzero and this can have performance implications, which need to be considered. When designing asynchronous digital circuits (i.e., circuits where information transfer is not dictated by a timekeeping signal such as a clock – note: all combinational circuits are asynchronous), attention must be paid to the possible presence of hazards. For a 5-minute extension read on hazards, please go through this site.

Consider again the MUX in Activity 1. Some extra signal names have been added in Figure 13 below.

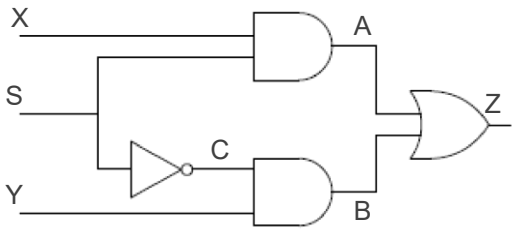


Figure 13: Schematic description of the 1-bit MUX, with intermediate signal names.

By inspection of the circuit, it is visually apparent that not all signal pathways cross the same number of gates, which means they will be affected by varying amounts of delay. Such a condition has the potential to create timing hazards, i.e., output glitches in particular operating conditions.

The Karnaugh map of the circuit (refer back to Table 2) can also confirm the likely presence of hazards. This circuit is implemented as a sum-of-products. 1-hazards can exist where any two adjacent 1s in the table are not covered by the same prime implicant loop. This is indeed the case for our MUX.

So, is this going to be an issue? You are about to find out.

Procedure

1. Create a new Vivado project called "mux_with_delays".
2. Create a new Verilog source named "my_mux". The logic formulation will be the same as you have used in Activity 1. However, the Verilog code will include intermediate signals, as shown in Code 3.

 © ANU 2025

Code 3

```
`timescale 1ns / 1ps

module my_mux(
    input wire X,
    input wire Y,
    output wire Z,
    input wire S
    );

wire A, B, C;
assign #5 C = (~S);
assign #5 A = X & S;
assign #5 B = Y & C;
assign #5 Z = A | B;
endmodule
```

Of note, the above code declares some extra `wire` entities for the purpose of describing **intermediate signals**. Furthermore, combinatorial `assign` statements have a #n parameter specified, which indicates the delay associated with that operation (n multiples of the specified timescale). Here, it is assumed that all gates have rise and fall times of 5 ns. It is possible to specify different rise and fall times if desired with the #(n_rise,n_fall) syntax.

Important: Delay parameters are only relevant in simulation. Once a design is implemented in hardware, actual signal delays will be determined by the physics of the microchip and cannot be set programmatically.

3. Import the testbench file used in Activity 1. **Make sure the option "Copy sources into project" is checked.**
4. Run a simulation and review the results. Identify the hazard. Under what change of inputs does it occur? How long does it last?
5. Let's dig a little deeper. In the Scope window, click on the UUT *my_mux* object. In the Objects window, you will see a list of all the module signals including internal signals. Drag and drop A, B, and C to the simulation window. Then re-launch the simulation using the loop button (see Figure 14) to compute results for all signals. Review the behaviour of the internal signals and make sure that you have understood what sequence of changes is causing the hazard. This simulation approach (i.e., visualising additional internal signals) is a very good troubleshooting approach for complex designs, so make sure you remember how to do it for the future.

 © ANU 2025

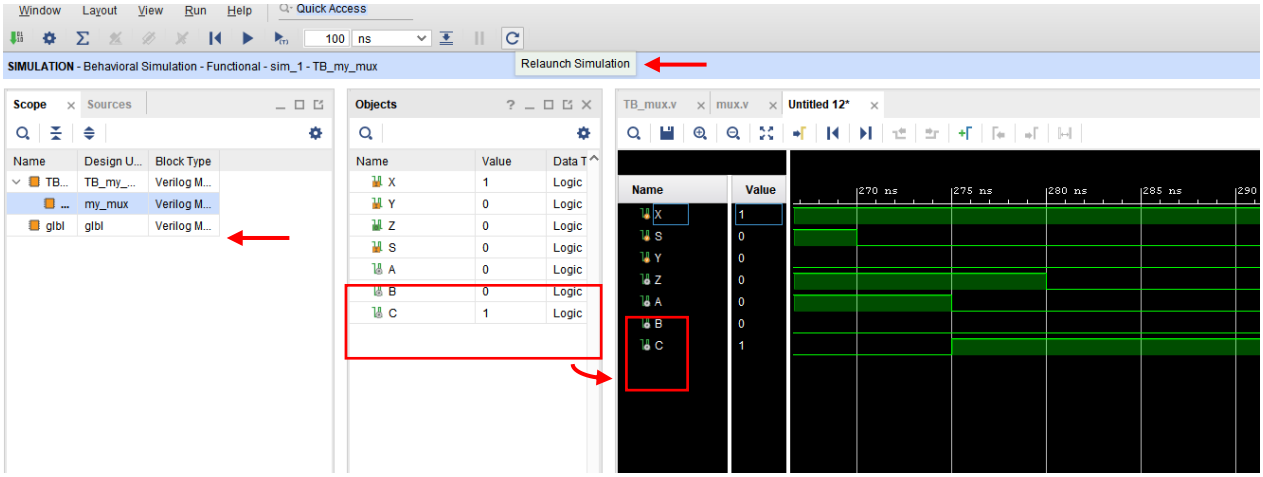


Figure 14: Simulation window. The Highlights show the list of additional internal signals (which can be dragged and dropped into the waveform display pane on the right) and the "relaunch simulation" button.

6. Assess the total transition time for the mux (i.e., the time it takes for a change in input to result in a stable output) for different signal transition events. Considering the gate delays and hazards, what is the maximum data rate (signal changes per second) that such a system can process successfully?

7. On the Karnaugh map (Table 4), the additional prime implicant to resolve the hazard condition is in the green box.

S \ XY	00	10	11	01
0	0	0	1	1
1	0	1	1	0

Table 4: Karnaugh Map of the 1-bit MUX with the additional prime implicant.

8. Determine the new SOP logic expression for the mux after the addition of the new prime implicant.

9. Modify the Verilog source accordingly. **You will need a new intermediate signal of type `wire` for the additional prime implicant.** Ensure that you assign a 5ns delay to the new AND gate required for computing the additional prime implicant.

   Please use a three-input OR gate to compute the output Z. The delay to the OR gate is 5ns.

   Note: All internal (intermediate) signals should be declared.

10. Run a new behavioural simulation and verify that the hazard has now disappeared. Review the maximum operating data rate under these new conditions and discover…

    …that it has NOT changed.

 © ANU 2025

Comment: The unchanged data rate performance result may seem confusing at first, but does make sense upon greater scrutiny. The concept of "maximum data rate" of a system implies changing an input and then capturing the output after enough time for transients to have exhausted. Glitches from hazards occur as part of transient behaviour and so would also be exhausted within this time, hence the unchanged result.

But wait, there is more!

a. The concept of periodic sampling is more appropriate for synchronous circuits, which only respond to signals at regular time intervals. In a truly asynchronous design, hazards are always bad, since asynchronous systems respond immediately to any signal changes, no matter how short-lived they may be.

b. In practice, the addition of extra logic to remove the hazard would likely slow down a circuit and can introduce new hazards.

11. In your notebook, note down the mux logic with and without the additional gate. Also, copy the key result traces and annotate where the hazard is present, vs removed, as well as the time interval used to calculate the maximum data rate. Show your work to the lab demonstrators to receive your grade. Please note, both simulation results must show all internal signals.

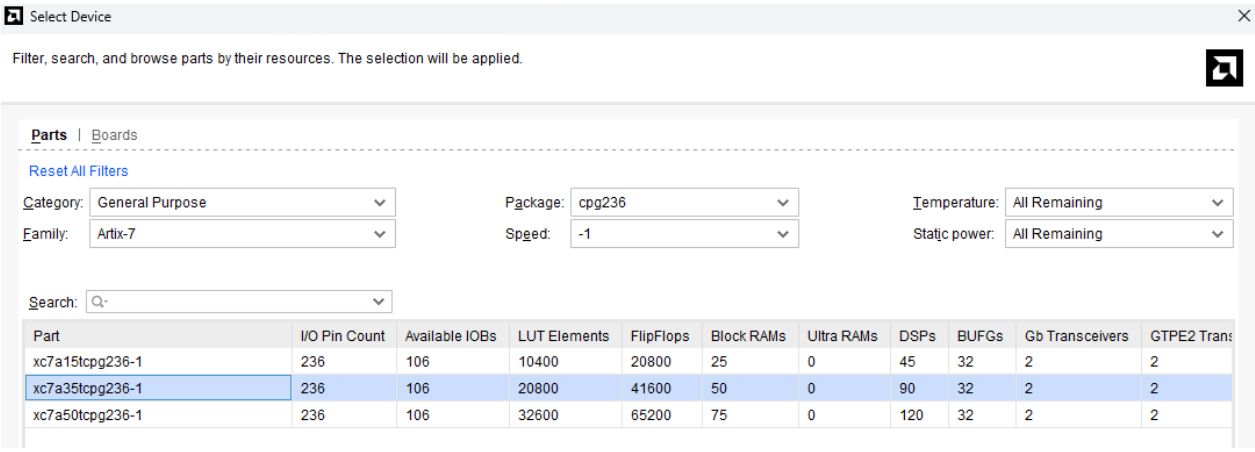
End of Lab One

 © ANU 2025

Appendix

Alternative part selection if using a Basys3 development board:

Development Board	Basys3
FPGA Family	Artix-7
Package	CPG236
Speed grade	-1
Temperature grade	All Remaining
Part number	xc7a35tcpg236-1



 © ANU 2025